

Recursive Partitioning, Information Theory, and Non-Parametric Decision Surfaces

A Unified Mathematical, Information-Theoretic, and Physical Foundation of
Decision Trees (CART)

Contents

1	The Recursive Space-Partitioning Paradigm	2
1.1	Top-Down Greedy Recursive Binary Splitting	2
2	Splitting Criteria for Classification Trees	2
2.1	Shannon Entropy and Information Gain	3
2.2	Gini Impurity	3
2.3	Misclassification Error	3
3	Splitting Criteria for Regression Trees	3
3.1	Mean Squared Error (MSE) / Variance Reduction	4
3.2	Mean Absolute Error (MAE) / Median Response	4
4	Geometric Characterization of Decision Surfaces	4
4.1	Orthogonal Hyperplane Constraints	4
5	Regularization and Cost-Complexity Pruning	5
5.1	Pre-Pruning (Early Stopping Rules)	5
5.2	Post-Pruning: Minimal Cost-Complexity Pruning	5
5.3	Weakest-Link Pruning Algorithm	6
6	Physical Analogies: Thermodynamic Entropy and Phase Separation	6
6.1	Thermodynamic Entropy of Mixing	6
6.2	Step-Potential Discretization of Energy Landscapes	7
7	Production-Grade Vectorized NumPy Implementation	7

1 The Recursive Space-Partitioning Paradigm

Decision trees, popularized through the **Classification and Regression Trees (CART)** methodology (Breiman et al., 1984), represent a class of non-parametric supervised learning models that partition feature space $\mathcal{X} \subseteq \mathbb{R}^d$ into a set of disjoint, axis-aligned hyper-rectangular regions $\mathcal{R} = \{R_1, R_2, \dots, R_M\}$.

Definition 1.1 (Disjoint Feature Space Partition). A tree-structured partition satisfies two structural properties over the input domain $\mathcal{X}$:

1. **Mutual Exclusivity:** $R_m \cap R_l = \emptyset \quad \forall m \neq l$.
2. **Exhaustive Coverage:** $\bigcup_{m=1}^M R_m = \mathcal{X}$.

Given an input feature vector $\mathbf{x} \in \mathcal{X}$, the decision tree maps $\mathbf{x}$ to a single terminal region (leaf node) R_m and assigns a localized constant prediction c_m :

$$f(\mathbf{x}) = \sum_{m=1}^M c_m \cdot \mathbb{I}(\mathbf{x} \in R_m) \quad (1)$$

where $\mathbb{I}(\cdot)$ is the indicator function.

1.1 Top-Down Greedy Recursive Binary Splitting

Finding the globally optimal tree partition that minimizes empirical risk across all possible tree topologies is an NP-complete combinatorial optimization problem. Practical decision tree induction relies on a **top-down, greedy recursive binary splitting algorithm** (a heuristic search).

Starting with the full training dataset $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$, the algorithm considers a split at node m along a single feature dimension $j \in \{1, 2, \dots, d\}$ at threshold $s \in \mathbb{R}$. This threshold splits region R_m into two half-spaces:

$$R_{m,L}(j, s) = \{\mathbf{x} \in R_m \mid x_j \leq s\}, \quad R_{m,R}(j, s) = \{\mathbf{x} \in R_m \mid x_j > s\} \quad (2)$$

The optimal split parameter pair (j^*, s^*) at node m minimizes the weighted sum of child node impurities:

$$(j^*, s^*) = \arg \min_{j \in \{1, \dots, d\}, s} \left[\frac{N_{mL}}{N_m} H(R_{m,L}(j, s)) + \frac{N_{mR}}{N_m} H(R_{m,R}(j, s)) \right] \quad (3)$$

where N_m is the sample count in region R_m , N_{mL} and N_{mR} are sample counts in the proposed left and right child regions, and $H(\cdot)$ is an impurity evaluation metric.

2 Splitting Criteria for Classification Trees

For classification tasks with categorical labels $y \in \{1, 2, \dots, C\}$, let N_m denote the total number of training observations in region R_m , and let N_{mc} denote the number of observations in R_m belonging to class c :

$$p_{mc} = \frac{1}{N_m} \sum_{\mathbf{x}^{(i)} \in R_m} \mathbb{I}(y^{(i)} = c) \quad (4)$$

The term p_{mc} represents the empirical conditional probability distribution over classes $c \in \{1, \dots, C\}$ within node m . The localized prediction for node m is the majority vote winner: $c_m = \arg \max_c p_{mc}$.

Node impurity measures $H(R_m)$ evaluate the heterogeneity of class distribution $\mathbf{p}_m = [p_{m1}, \dots, p_{mC}]^T$.

2.1 Shannon Entropy and Information Gain

Derived from information theory (Shannon, 1948), ****Node Entropy**** measures the average information deficit or uncertainty contained within node m :

Definition 2.1 (Shannon Entropy Impurity).

$$H_{\text{Entropy}}(R_m) = - \sum_{c=1}^C p_{mc} \log_2(p_{mc}) \quad (5)$$

with the convention $0 \log_2 0 = 0$.

Evaluating a split (j, s) using entropy yields the ****Information Gain**** $\Delta H(j, s)$, which quantifies the reduction in entropy (uncertainty) achieved by partitioning region R_m :

$$\Delta H(j, s) = H(R_m) - \left[\frac{N_{mL}}{N_m} H(R_{m,L}) + \frac{N_{mR}}{N_m} H(R_{m,R}) \right] \quad (6)$$

Maximizing Information Gain $\Delta H(j, s)$ is mathematically equivalent to minimizing the weighted child node entropy.

2.2 Gini Impurity

The ****Gini Impurity**** measures the expected error rate if a randomly selected element from node m were randomly labeled according to the empirical class distribution $\mathbf{p}_m$:

Definition 2.2 (Gini Impurity).

$$H_{\text{Gini}}(R_m) = \sum_{c=1}^C p_{mc}(1 - p_{mc}) = 1 - \sum_{c=1}^C p_{mc}^2 \quad (7)$$

For binary classification ($C = 2$) where $p_{m1} = p$ and $p_{m2} = 1 - p$:

$$H_{\text{Gini}}(p) = p(1 - p) + (1 - p)p = 2p(1 - p) \quad (8)$$

$$H_{\text{Entropy}}(p) = -p \log_2(p) - (1 - p) \log_2(1 - p) \quad (9)$$

2.3 Misclassification Error

The direct ****Misclassification Error**** rate represents the empirical classification risk under a zero-one loss function:

$$H_{\text{Error}}(R_m) = 1 - \max_{c \in \{1, \dots, C\}} p_{mc} \quad (10)$$

Property 2.1 (Strict Concavity and Tree Growth). Misclassification error $H_{\text{Error}}(R_m)$ is continuous but not strictly concave. Because of this, splitting a node can leave the weighted misclassification error unchanged, causing greedy search algorithms to stall.

In contrast, Shannon Entropy H_{Entropy} and Gini Impurity H_{Gini} are ****strictly concave functions**** over the simplex $\sum p_{mc} = 1$. Strict concavity guarantees that any split producing distinct class probabilities in child nodes strictly reduces overall node impurity ($\Delta H > 0$), driving continuous tree growth.

3 Splitting Criteria for Regression Trees

For regression tasks with continuous targets $y \in \mathbb{R}$, a decision tree models the conditional expectation $\hat{y}(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]$.

Within terminal region R_m , the optimal constant response $\hat{c}_m$ minimizing Mean Squared Error (MSE) is the empirical mean of the targets in R_m :

$$\hat{c}_m = \frac{1}{N_m} \sum_{\mathbf{x}^{(i)} \in R_m} y^{(i)} \quad (11)$$

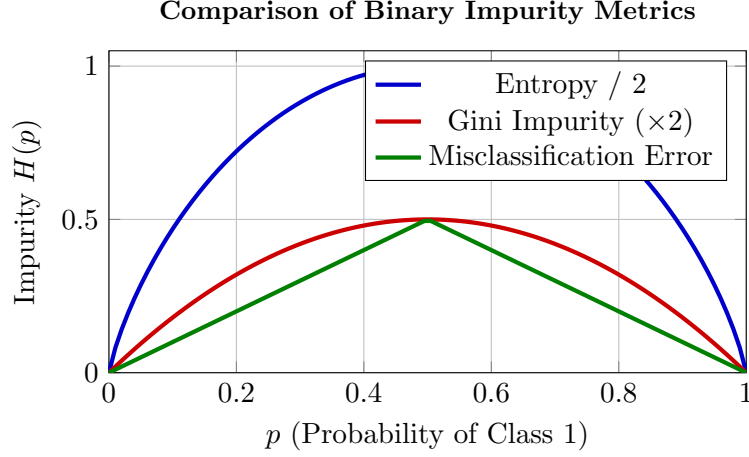


Figure 1: Normalized comparison of impurity metrics for binary classification. Gini and Entropy are strictly concave, penalizing mixed nodes more heavily than linear error.

3.1 Mean Squared Error (MSE) / Variance Reduction

The standard impurity metric for regression trees is the **Node Variance** (Mean Squared Error):

Definition 3.1 (MSE Node Impurity).

$$H_{\text{MSE}}(R_m) = \frac{1}{N_m} \sum_{\mathbf{x}^{(i)} \in R_m} \left(y^{(i)} - \hat{c}_m \right)^2 \quad (12)$$

The evaluation criterion for a split (j, s) corresponds to maximizing **Variance Reduction**:

$$\Delta H_{\text{MSE}}(j, s) = H_{\text{MSE}}(R_m) - \left[\frac{N_{mL}}{N_m} H_{\text{MSE}}(R_{m,L}) + \frac{N_{mR}}{N_m} H_{\text{MSE}}(R_{m,R}) \right] \quad (13)$$

3.2 Mean Absolute Error (MAE) / Median Response

When target distributions contain extreme outliers, minimizing MSE forces splits to isolate individual noise points. To improve robustness, we substitute L_1 loss (Mean Absolute Error).

Under L_1 loss, the optimal constant response $\hat{c}_m$ within region R_m is the empirical **median** of the target values:

$$\hat{c}_m = \text{median} \left(\{y^{(i)} \mid \mathbf{x}^{(i)} \in R_m\} \right) \quad (14)$$

Definition 3.2 (MAE Node Impurity).

$$H_{\text{MAE}}(R_m) = \frac{1}{N_m} \sum_{\mathbf{x}^{(i)} \in R_m} \left| y^{(i)} - \hat{c}_m \right| \quad (15)$$

4 Geometric Characterization of Decision Surfaces

Decision trees partition feature space using axis-aligned hyperplanes orthogonal to coordinate axes.

4.1 Orthogonal Hyperplane Constraints

Every decision rule at an internal node m has the mathematical form:

$$\mathbf{e}_j^T \mathbf{x} - s = 0 \quad (16)$$

where $\mathbf{e}_j = [0, \dots, 1, \dots, 0]^T$ is the standard basis vector along feature dimension j .

Property 4.1 (Coordinate Orientation Sensitivity). Because every split boundary is strictly parallel to a coordinate axis, decision trees are inherently sensitive to feature rotations.

If a simple, linearly separable decision boundary lies at a 45° angle relative to feature axes ($\mathbf{w}^T \mathbf{x} = 0$ with $\mathbf{w} = [1, 1]^T$), a linear model isolates the boundary with a single parameter. A decision tree must approximate this diagonal line using a complex, multi-step "staircase" approximation, requiring deep trees and large sample sizes.

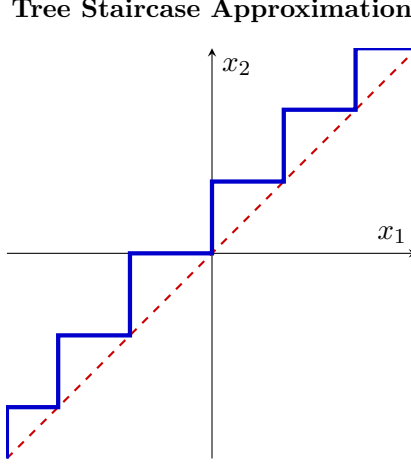


Figure 2: A decision tree approximating a smooth diagonal boundary ($x_2 = x_1$) using axis-aligned orthogonal splits, creating a step-wise staircase decision surface.

5 Regularization and Cost-Complexity Pruning

Fully grown decision trees expand until terminal nodes are pure ($H = 0$), overfitting training noise and producing high variance. Regularization strategies control tree complexity to balance the bias-variance tradeoff.

5.1 Pre-Pruning (Early Stopping Rules)

Pre-pruning restricts tree growth during construction using heuristic threshold constraints:

- **Maximum Depth ($d_{\max}$):** Halts recursive splitting when node depth reaches $d_{\max}$.
- **Minimum Sample Split (N_{split}):** Prevents node splitting if sample count $N_m < N_{\text{split}}$.
- **Minimum Information Gain ($\Delta H_{\min}$):** Blocks splits yielding impurity reductions below $\Delta H_{\min}$.

While computationally efficient, pre-pruning suffers from **shortsightedness** (the horizon effect): a split with low initial Information Gain may enable valuable downstream splits.

5.2 Post-Pruning: Minimal Cost-Complexity Pruning

Post-pruning constructs a fully grown tree T_0 first, then removes non-essential subtrees using structural risk minimization.

Let $T \subset T_0$ represent a pruned subtree, $|\tilde{T}|$ denote the number of terminal leaf nodes in T , and R_m represent the empirical risk within leaf m . The overall empirical risk of subtree T is:

$$R(T) = \sum_{m=1}^{|\tilde{T}|} N_m H(R_m) \quad (17)$$

Definition 5.1 (Cost-Complexity Objective). For a complexity regularization parameter $\alpha \geq 0$, the **Cost-Complexity Risk** $R_\alpha(T)$ is:

$$R_\alpha(T) = R(T) + \alpha|\tilde{T}| = \sum_{m=1}^{|\tilde{T}|} N_m H(R_m) + \alpha|\tilde{T}| \quad (18)$$

- $\alpha = 0$: Unpenalized objective, selecting the fully grown tree T_0 .
- $\alpha \rightarrow \infty$: Heavy penalty on complexity, collapsing the tree down to a single root node ($|\tilde{T}| = 1$).

5.3 Weakest-Link Pruning Algorithm

For any internal node t supporting a subtree T_t , collapsing T_t into a single node t increases empirical risk by $R(t) - R(T_t)$ while reducing leaf node count by $|\tilde{T}_t| - 1$.

Setting the cost-complexity of the unpruned subtree equal to the collapsed single node identifies the critical regularization threshold $\alpha(t)$:

$$R(T_t) + \alpha|\tilde{T}_t| = R(t) + \alpha(1) \implies \alpha(t) = \frac{R(t) - R(T_t)}{|\tilde{T}_t| - 1} \quad (19)$$

The **Weakest-Link Pruning** algorithm proceeds as follows: . Compute $\alpha(t)$ for every internal node $t \in T_0$.

Identify the "weakest link" node t^* that minimizes $\alpha(t)$: $t^* = \arg \min_t \alpha(t)$.

Collapse subtree T_{t^*} into a single leaf node to produce a smaller tree T_1 .

Repeat recursively to generate a finite sequence of nested subtrees: $T_0 \supset T_1 \supset T_2 \cdots \supset T_{\text{root}}$.

Select the optimal subtree T_α^* from this sequence using K -fold cross-validation.

6 Physical Analogies: Thermodynamic Entropy and Phase Separation

Decision tree mechanics map onto statistical thermodynamics, phase transitions, and potential energy discretizations.

6.1 Thermodynamic Entropy of Mixing

Consider a multi-component chemical fluid containing C molecular species with mole fractions p_c . The **Thermodynamic Entropy of Mixing** ΔS_{mix} per particle is:

$$\Delta S_{\text{mix}} = -k_B \sum_{c=1}^C p_c \ln(p_c) \quad (20)$$

where k_B is the Boltzmann constant.

* **High Entropy State (Unsplit Root Node):** Represents a homogeneous, well-mixed phase where all chemical species are uniformly intermixed ($p_c \approx 1/C$). Thermodynamic entropy ΔS_{mix} is maximized, representing maximum disorder. * **Low Entropy State (Pure Leaf Nodes):** Represents complete **Phase Separation** (spinodal decomposition) where chemical components separate into distinct pure phases ($p_c = 1$). Node entropy collapses to zero.

Decision tree splitting acts as a **Maxwell's Demon**, using feature thresholds as sorting barriers to sort mixed particles into pure physical phases, minimizing total mixing entropy.

6.2 Step-Potential Discretization of Energy Landscapes

In quantum mechanics and molecular dynamics, continuous potential energy surfaces $V(\mathbf{x})$ are often approximated using **Piecewise-Constant Step Potentials**:

$$V(\mathbf{x}) \approx \sum_{m=1}^M V_m \cdot \mathbb{I}(\mathbf{x} \in R_m) \quad (21)$$

Regression trees perform an identical operation. They approximate smooth, multi-dimensional response surfaces $y = f(\mathbf{x}) + \epsilon$ by discretizing feature space into flat energy plateaus (c_m), converting continuous differential operations into discrete step transitions across region boundaries.

7 Production-Grade Vectorized NumPy Implementation

The following Python code implements a production-grade decision tree framework supporting both classification (Gini / Entropy) and regression (MSE), complete with pre-pruning constraints.

```
1 import numpy as np
2
3 class Node:
4     """Represents a single node within the decision tree structure."""
5     def __init__(self, feature=None, threshold=None, left=None, right=None,
6         *, value=None, impurity=None):
7         self.feature = feature          # Feature index along which split
8         self.threshold = threshold      # Threshold value for split
9         self.left = left                # Left child Node object
10        self.right = right               # Right child Node object
11        self.value = value               # Prediction value if terminal leaf
12        self.impurity = impurity         # Impurity score at node
13
14    @property
15    def is_leaf(self) -> bool:
16        return self.value is not None
17
18 class UniversalDecisionTreeEngine:
19     """
20     Production-grade Decision Tree (CART) Engine supporting both
21     Classification
22     (Gini, Entropy) and Regression (MSE), built with vectorized NumPy
23     operations.
24     """
25     def __init__(self, criterion: str = 'gini', max_depth: int = 10,
26         min_samples_split: int = 2, min_impurity_decrease: float =
27         1e-7,
28         mode: str = 'classification'):
29         self.criterion = criterion
30         self.max_depth = max_depth
31         self.min_samples_split = min_samples_split
32         self.min_impurity_decrease = min_impurity_decrease
33         self.mode = mode
34
35         self.root = None
36         self.n_classes_ = None
37
38     def _calculate_impurity(self, y: np.ndarray) -> float:
39         """Calculates node impurity based on task mode and criterion."""
```

```

37     N = len(y)
38     if N == 0:
39         return 0.0
40
41     if self.mode == 'classification':
42         counts = np.bincount(y)
43         probs = counts / N
44         probs = probs[probs > 0]
45
46         if self.criterion == 'gini':
47             return 1.0 - np.sum(probs**2)
48         elif self.criterion == 'entropy':
49             return -np.sum(probs * np.log2(probs))
50         else:
51             raise ValueError(f"Unknown classification criterion: {self.
criterion}")
52
53     elif self.mode == 'regression':
54         if self.criterion == 'mse':
55             mean_y = np.mean(y)
56             return np.mean((y - mean_y)**2)
57         else:
58             raise ValueError(f"Unknown regression criterion: {self.
criterion}")
59
60     def _calculate_leaf_value(self, y: np.ndarray):
61         """Computes optimal terminal leaf node prediction."""
62         if self.mode == 'classification':
63             # Majority vote winner
64             return np.argmax(np.bincount(y))
65         elif self.mode == 'regression':
66             # Empirical mean response
67             return np.mean(y)
68
69     def _best_split(self, X: np.ndarray, y: np.ndarray):
70         """
71         Locates optimal (feature, threshold) split pair maximizing impurity
72         reduction
73         using vectorized threshold evaluations.
74         """
75         N, d = X.shape
76         best_gain = -1.0
77         split_idx, split_thresh = None, None
78
79         parent_impurity = self._calculate_impurity(y)
80
81         for feat_idx in range(d):
82             X_column = X[:, feat_idx]
83             # Unique sorted candidate thresholds
84             sort_indices = np.argsort(X_column)
85             X_sorted, y_sorted = X_column[sort_indices], y[sort_indices]
86
87             # Identify candidate split thresholds (midpoints between
88             adjacent values)
89             split_mask = X_sorted[:-1] != X_sorted[1:]
90             if not np.any(split_mask):
91                 continue
92
93             thresholds = (X_sorted[:-1][split_mask] + X_sorted[1:][
split_mask]) / 2.0

```

```

92         for thresh in thresholds:
93             left_mask = X_column <= thresh
94             right_mask = ~left_mask
95
96             N_L, N_R = np.sum(left_mask), np.sum(right_mask)
97             if N_L < 1 or N_R < 1:
98                 continue
99
100             impurity_L = self._calculate_impurity(y[left_mask])
101             impurity_R = self._calculate_impurity(y[right_mask])
102
103             # Impurity decrease / Information Gain
104             child_impurity = (N_L / N) * impurity_L + (N_R / N) *
105 impurity_R
106             gain = parent_impurity - child_impurity
107
108             if gain > best_gain:
109                 best_gain = gain
110                 split_idx = feat_idx
111                 split_thresh = thresh
112
113             return split_idx, split_thresh, best_gain
114
115 def _build_tree(self, X: np.ndarray, y: np.ndarray, depth: int = 0) ->
Node:
116     """Recursively constructs decision tree hierarchy."""
117     N, d = X.shape
118     num_labels = len(np.unique(y))
119     current_impurity = self._calculate_impurity(y)
120
121     # Stopping criteria evaluation
122     if (depth >= self.max_depth or
123         N < self.min_samples_split or
124         num_labels == 1 or
125         current_impurity == 0.0):
126         leaf_val = self._calculate_leaf_value(y)
127         return Node(value=leaf_val, impurity=current_impurity)
128
129     # Locate best split
130     feat_idx, thresh, gain = self._best_split(X, y)
131
132     if feat_idx is None or gain < self.min_impurity_decrease:
133         leaf_val = self._calculate_leaf_value(y)
134         return Node(value=leaf_val, impurity=current_impurity)
135
136     # Split current node into left and right subtrees
137     left_mask = X[:, feat_idx] <= thresh
138     right_mask = ~left_mask
139
140     left_child = self._build_tree(X[left_mask], y[left_mask], depth +
1)
141     right_child = self._build_tree(X[right_mask], y[right_mask], depth
+ 1)
142
143     return Node(feature=feat_idx, threshold=thresh,
144                 left=left_child, right=right_child, impurity=
current_impurity)
145
146 def fit(self, X: np.ndarray, y: np.ndarray):

```

```

147     """Fits Decision Tree structure to training data."""
148     X = np.asarray(X, dtype=np.float64)
149     y = np.asarray(y)
150
151     if self.mode == 'classification':
152         self.n_classes_ = len(np.unique(y))
153         y = y.astype(int)
154
155     self.root = self._build_tree(X, y, depth=0)
156     return self
157
158     def _traverse_tree(self, x: np.ndarray, node: Node):
159         """Traverses tree hierarchy for a single test sample."""
160         if node.is_leaf:
161             return node.value
162
163         if x[node.feature] <= node.threshold:
164             return self._traverse_tree(x, node.left)
165         return self._traverse_tree(x, node.right)
166
167     def predict(self, X: np.ndarray) -> np.ndarray:
168         """Predicts outputs for query matrix X."""
169         X = np.asarray(X, dtype=np.float64)
170         return np.array([self._traverse_tree(x, self.root) for x in X])
171
172
173 if __name__ == "__main__":
174     np.random.seed(42)
175
176     # 1. Validation Run: Classification Mode
177     N_samples = 300
178     X_cls = np.random.randn(N_samples, 2)
179     # Non-linear XOR-like decision region
180     y_cls = ((X_cls[:, 0] > 0) ^ (X_cls[:, 1] > 0)).astype(int)
181
182     tree_clf = UniversalDecisionTreeEngine(criterion='gini', max_depth=5,
183 mode='classification')
184     tree_clf.fit(X_cls, y_cls)
185     preds_cls = tree_clf.predict(X_cls)
186
187     acc = np.mean(preds_cls == y_cls)
188     print(f"=== CLASSIFICATION TREE ACCURACY: {acc * 100:.2f}% ===")
189
190     # 2. Validation Run: Regression Mode
191     X_reg = np.sort(np.random.uniform(-3.0, 3.0, (N_samples, 1)), axis=0)
192     y_reg = np.sin(X_reg.ravel()) + 0.1 * np.random.randn(N_samples)
193
194     tree_reg = UniversalDecisionTreeEngine(criterion='mse', max_depth=4,
195 mode='regression')
196     tree_reg.fit(X_reg, y_reg)
197     preds_reg = tree_reg.predict(X_reg)
198
199     mse = np.mean((preds_reg - y_reg)**2)
200     print(f"=== REGRESSION TREE MEAN SQUARED ERROR: {mse:.4f} ===")

```

Listing 1: Vectorized NumPy implementation of CART Decision Trees for Classification and Regression.